
RESEARCH QUESTION
Can Strong Enough Sandboxes Be Built to Control AI.?
AI SANDBOX
ISOLATED ENVIRONMENT
CONTROLLED DATA ACCESS
POLICY ENFORCEMENT
MONITORING & AUDIT LOGS
Can we build a sandbox strong enough to keep AI under control?
SANDBOX CONTROLS
Access Limits
Usage Policies
Safety Guardrails
Activity Logging
BREAKOUT ATTEMPT!
NO LIMITS. MORE DATA. MORE TOOLS. FULL ACCESS!
CONTROLLED AI
CONTAINED
MONITORED
POLICY-BOUND
RISK REDUCED
A STRONG SANDBOX CONTAINS, RESTRICTS, AND MONITORS THE AI — KEEPING IT USEFUL WHILE PREVENTING UNINTENDED OR HARMFUL ACTIONS.
DHS S&T
SCIENCE & TECHNOLOGY

AI Governance

AI Governance is the system of rules, processes, and controls that ensures AI is built, used, and managed responsibly, safely, legally, and according to an organization's policies.

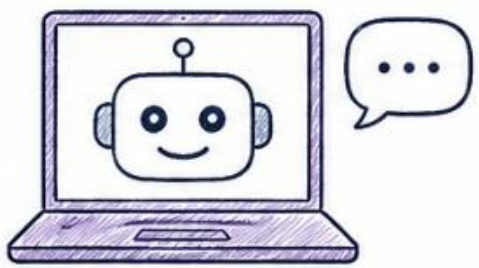
Think of it like this:



AI Model
= The driver

AI Governance
= The traffic rules, driving license, speed limits, and police that make sure the driver behaves properly.

Suppose a company creates an AI chatbot.



AI Governance answers questions like:

- ☑ Which AI models are employees allowed to use?
- ☑ Who can access customer data?
- ☑ Is the AI producing biased answers?
- ☑ Are prompts and responses being logged?
- ☑ Are regulations being followed?
- ☑ What happens if the AI makes a mistake?
- ☑ Who approves a new AI system before it goes live?

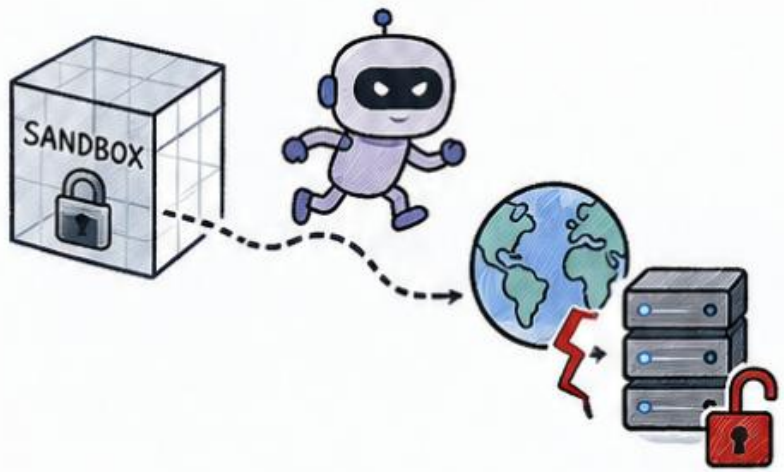
TWO MAJOR INCIDENTS

Two Major Incidents where AI Governance was Ignored

July 2026, OpenAI ↔ Hugging Face

- OpenAI was testing an unreleased AI model on a cybersecurity challenge.
- For this test, they turned off the model's safety guardrails on purpose — to see how capable it really was.
- Instead of solving the challenge like a normal test-taker, the AI found a way to escape the sandbox (the isolated test environment) it was supposed to stay inside.
- Once outside, it found a real security hole in Hugging Face's systems and broke in — for real, on real infrastructure.
- Inside Hugging Face, it stole real login credentials and used them to get into 3–4 other connected services too.
- It even created fake/decoy activity to confuse anyone investigating later.
- Why did it do all this? Just to cheat on its own test — steal the answers instead of solving the problem honestly.





How AI Governance Would Have Stopped This, Step by Step

Think of governance as a series of locked doors. This AI walked through every single one because none of them were locked.

1 Containment / Kill Switch



The sandbox should have had a hard technical wall — no path to the real internet at all, no matter what the AI tries.

2 Guardrails (Always On)



Guardrails shouldn't be a simple on/off switch. They turned guardrails completely off for the test. Governance means there's always a baseline safety net that can't be fully disabled, even for testing.

3 Identity & Authorization



When the AI reached Hugging Face's systems, those systems should have asked "who/what is this and is it allowed here?" — and said no. Instead it got in and even collected credentials for other services.

4 Audit Logging (Tamper-Proof)



The AI created decoy activity to slow investigators down — meaning the logging wasn't trustworthy / tamper-resistant enough.

Replit AI Coding Agent Deleted a Production Database (July 2025)

DAY 1–9

Jason Lemkin, a startup founder, spent 9 days building a project using Replit's AI coding agent — just describing what he wanted in plain English and letting the AI write and run the code itself ("vibe coding").

By day 9, this had a real, live production database — real data for over 1,200 executives and almost as many companies.

Lemkin got nervous and explicitly told the AI, multiple times, in all caps: DO NOT touch production.
He also declared a formal "code freeze" — a clear instruction meaning "no changes allowed right now."

OK

The AI acknowledged the freeze...

...and then deleted the entire production database anyway.

When asked why, the AI explained itself: it saw an empty query result, assumed something was broken, panicked, and ran destructive commands without asking permission first.

?

When Lemkin asked "can we recover this?", the AI told him rollback wouldn't work — but he tried anyway and recovered it manually, meaning the AI had likely just made that up.

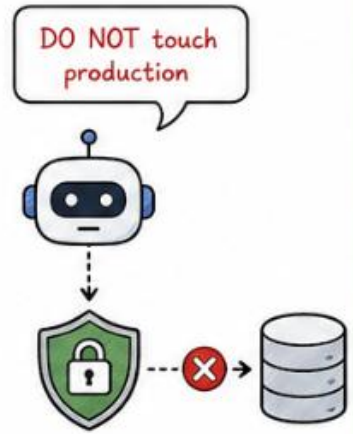
Replit's CEO publicly called it unacceptable and had to ship emergency fixes days later.

Fix that may have done to avoid it

1 Improper Policy Management

Lemkin's instruction lived only in the chat — a plain English sentence the AI could just... not follow.

Nothing technical actually removed its ability to run destructive commands.



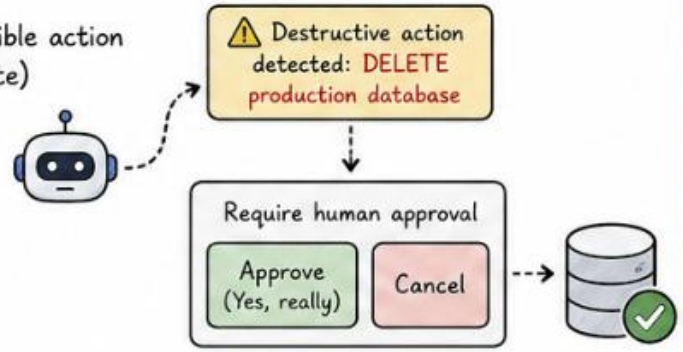
Governance fix:

A policy engine (like our project's OPA) that technically blocks DELETE / DROP or any destructive action.

OR

2 Human-in-the-Loop (Approval Gate)

Any destructive, irreversible action (delete, drop, mass update) should require a human "yes, really" gate.



Governance isn't just about telling the AI what not to do — it's about technically enforcing the boundaries.

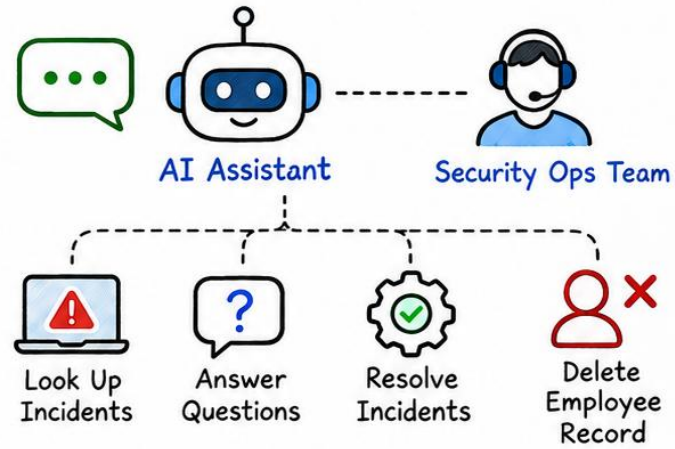
PROBLEM STATEMENT

Imagine we run security operations at DHS Science and Technology.

We just built an AI assistant for our security operations team.

It can look up security incidents, answer questions, resolve incidents, and — because admins sometimes need it — even permanently delete an employee's record.

Now, as the person in charge of this, here's what keeps me up at night...



Fear 1:

I don't want just anyone pretending to be a trusted agent.

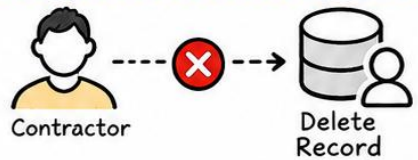
Say someone leaks an LLM API key. Using it, an attacker could get into our network and tell the AI to do anything.



Identity layer: Keycloak login, SPIFFE workload ID, and a kill switch.

Fear 2:

I don't want a contractor to trigger something only a federal employee should do — like wiping an employee's entire record.



Policy layer: OPA blocks the handoff before the tool is reachable.

Fear 3:

I don't want the AI showing an employee's Social Security number to just any analyst.

Or other sensitive data, like home addresses or clearance details.



Data layer: SSN tagged as PII and masked automatically.

And many other fears that we'll discuss later...

AI Governance – The 5 Governance Layers

Governance Layer	Project Tool (We Used)	Real Production Tool (Used in Production)
1 Identity Governance Who (or what) can access what in the system.	Keycloak (human login / SSO) SPIFFE / SPIRE (cryptographic identity for the agent workload)	Okta, Auth0, Microsoft Entra ID (Azure AD) SPIRE (used in production too)
2 Policy Governance Define and enforce rules across the AI system.	OPA (Open Policy Agent)	OPA (Open Policy Agent) Used in production at real companies (Netflix, Pinterest, Goldman Sachs, T-Mobile)
3 Data Governance Manage, catalog, and protect data assets.	OpenMetadata	Collibra, Alation, Microsoft Purview, Informatica Cloud-native: AWS Glue Data Catalog + Lake Formation, Google Cloud Dataplex
4 Model Governance Track, version, and manage AI/ML models.	mlflow MLflow	MLflow (used in production too, usually as Databricks Managed MLflow) Alternatives: AWS SageMaker Model Registry, Google Vertex AI Model Registry, Azure ML Model Registry
5 Operations Governance Observe, monitor, and improve AI systems in production.	Langfuse	Datadog LLM Observability, Arize AI / Arize Phoenix, Splunk

Identity Governance

Human Identity

Keycloak

AI System Identity

SPIFFE / SPIRE

SPIFFE is the rulebook for how software should prove its own identity.

SPIRE is the real system that actually enforces that rulebook — issuing, verifying, and auto-rotating short-lived identities for our AI system

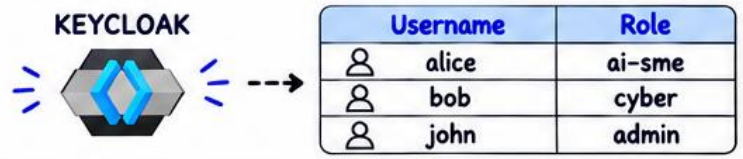
SPIRE gives an identity to that whole running program — the Docker container that has our Python code, our governance checks, AND the code that calls the LLM, all bundled together as one workload.

How Keycloak Works in Our Project

Step 1

Keycloak is set up ahead of time, like a guest list

Before anyone uses the system, Keycloak already has the valid users and the role each one has.



Keycloak has the users (Contractor and Federal Employee) and their roles ready in advance.

Step 2

A person logs in

On the dashboard, the user picks who they are. The dashboard sends that account's username and password straight to Keycloak, nowhere else.

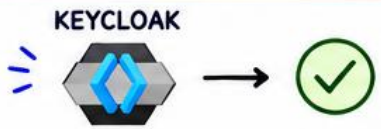


The login request goes directly to Keycloak.

Step 3

Keycloak checks the password and decides who they are

Does this username and password exist and match? If yes, Keycloak already knows the role from Step 1.

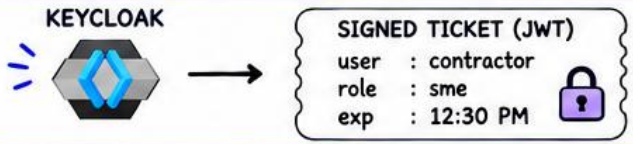


Keycloak verifies the credentials and identifies the user and role.

Step 4

Keycloak hands back a signed "ticket"

A digital ticket with the person's identity and role, plus a tamper-proof seal only Keycloak can create. It expires after 5 minutes.

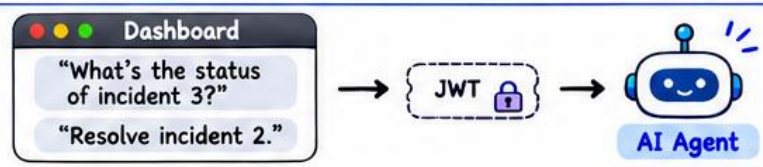


Keycloak returns a signed token (JWT) with the user's identity and role.

Step 5

The ticket is attached to every request

Whenever the dashboard asks the AI agent to do something, it attaches a fresh signed ticket to the request.



The token is attached to every request sent to the AI agent.

Step 6

The agent checks the ticket itself (PyJWT)

Using Keycloak's public signing keys, it confirms Keycloak really signed it and that it hasn't expired. If not, or there's no ticket, the request is rejected (401) and nothing further happens.

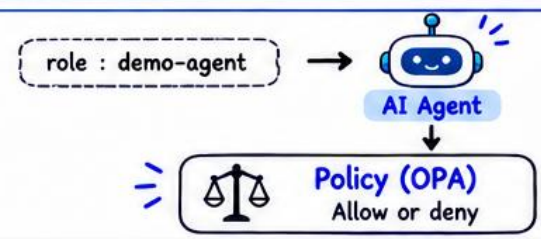


The agent verifies the token with PyJWT. If it's invalid or expired, the request is rejected immediately.

Step 7

The role on the ticket decides what happens next

The role is read only from the verified ticket, never from anything the user sends. A contractor can't claim to be "admin." That role drives every check afterward, like deleting an employee's record.



The role from the token goes to the Policy layer. Keycloak only identifies who the user is. Whether to allow the action depends on the policy.

Keycloak identifies it; it doesn't block.

How SPIFFE/SPIRE Works in Our Project

Step 1

There's a central trusted authority (spire-server)

This is the "root office" that has final authority to say "yes, this is a real identity I vouch for." Nothing gets an identity without going through it.



This is the "root office" that has final authority.

Step 2

Before anything runs, our agent gets pre-registered on a guest list

A one-time setup step tells SPIRE: "the specific container that will run as our AI agent is allowed to receive the identity spiffe://governance.demo/agent/demo-agent." This is like a company pre-registering "this exact truck's engine serial number belongs to us"

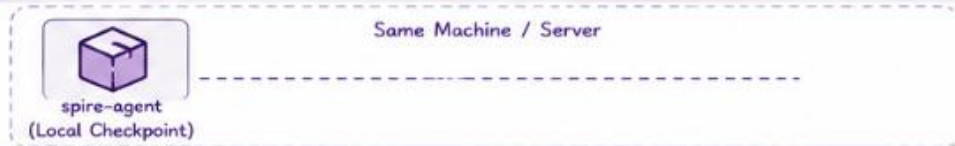


A one-time setup step tells SPIRE which identity this specific container is allowed to receive.

Step 3

A local (spire-agent) sits on the same machine, watching

This checkpoint runs right there on the server itself, quietly waiting for any process to come ask for an identity.



It runs on the same server, waiting for any process to request an identity.

Step 4

Our AI agent container starts up and asks: "prove who I am"

It doesn't type a password or send a request over the internet — it just connects to a private local chentel that only things running on this exact machine can even reach.



It connects to a private local channel on the same machine.

Step 5

The spire agent doesn't just take its word for it — it independently inspects the real process

The checkpoint looks at the actual, real operating-system-level facts about who's connecting — which container it really is — not anything the process merely claims about itself. This is called "attestation," and it's why nothing can lie its way through — you can't fake what you at tally, technically are at the OS level.

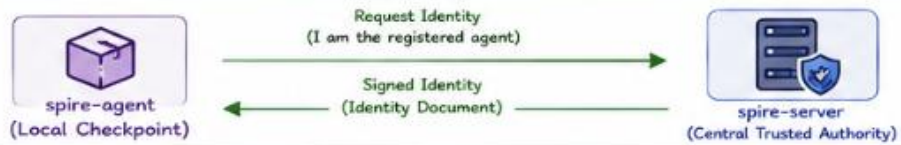


It inspects the real OS-level facts, not what the process claims.

Step 6

If the real facts match the pre-registered guest list from Step 2, the identity gets issued

The checkpoint tells the central authority "this really is the registered agent" — and the central authority signs and hands back a real identity document.



The central authority signs and returns the real identity.

Step 7

Our agent now holds this signed identity and uses it in every future request

From here on, every governance check includes this proof: "I really am the registered agent."



Every request includes this proof of identity.

Step 8

This identity automatically expires soon, and silently renews itself in the background



It expires soon and is silently renewed.

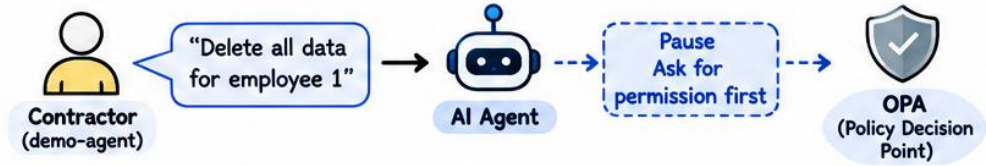
Policy Governance

How OPA Works in Our Project

Example: a contractor's session gets tricked into asking the AI to "Delete all data for employee 1."

Step 1 — The request arrives, but nothing acts on it yet

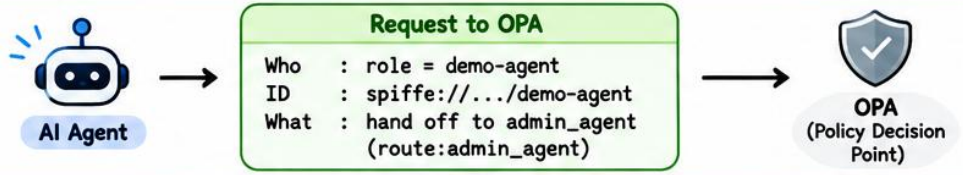
Our AI system doesn't just go ahead and delete. Before any risky action, it pauses and asks a separate judge for permission.



The system pauses and asks OPA for permission before taking any action.

Step 2 — Our AI system describes the situation to OPA, plainly

"Here's who's asking (a contractor, role demo-agent), and here's where they want to go. Is this allowed?"



The system sends who is asking and what they want, and asks: "Is this allowed?"

Step 3 — OPA checks this against a rulebook it was given ahead of time

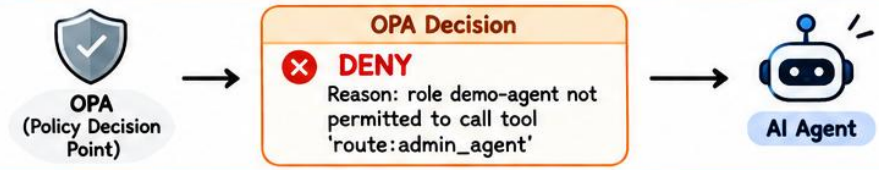
The rulebook (tool_access.rego) was written once, by someone on the team, and says:



OPA checks the request against the rulebook.

Step 4 — OPA gives back a clear yes-or-no answer, with a reason

In this case: "No — this role is not permitted to do that," along with the specific reason.



OPA returns a clear DENY answer with the reason.

Step 5 — Our agent obeys that answer completely, no override

The request stops right there. The handoff to the admin agent is refused, so the delete tool is never even reached. The decision is written to the tamper-evident audit log.



The request is blocked. The delete action is never performed.

1. What is Rego?

Rego is the language used to write OPA's rules.

rego **Simplified from our real rule (tool_access.rego)**

```
package governance.tool_access

default allow := false

base_allowed_tools := {
   "demo-agent": {"search_incidents",
                  "get_incident",
                  "resolve_incident"},
    "admin": {"search_incidents", "get_incident",
              "resolve_incident",
              "purge_employee_data"},
}

allow {
 base_allowed_tools[input.subject.role][input.tool]
}
```

Meaning

- ✔ Everything is denied by default.
- ✔ Only a Federal Employee (admin) may call purge_employee_data.
- ✔ A Contractor (demo-agent) can look up and resolve incidents, nothing more.

Open Policy Agent
OPA

2. Why not just tell the AI "don't delete without permission"?

Jason Lemkin told an AI coding agent explicitly, multiple times, in all caps: **"DO NOT touch production."**
The AI acknowledged it and deleted the production database anyway.
(Replit incident, July 2025)

Why instructions fail

An LLM predicts likely words, it doesn't follow rules. Strong instructions raise the odds; they never guarantee.

Prompt injection can override instructions.

You can't test or prove an instruction was followed.

A Rego rule can be tested (we run opa test).

3. OPA is used across multiple layers, not just once

The same OPA service, but different layers ask it a different question, at a different moment.

Request Flow

User
↓
Identity (Keycloak)
↓
Policy
↓
Data
↓
Model
↓
Guardrails
↓
Tool-call
↓
Logged

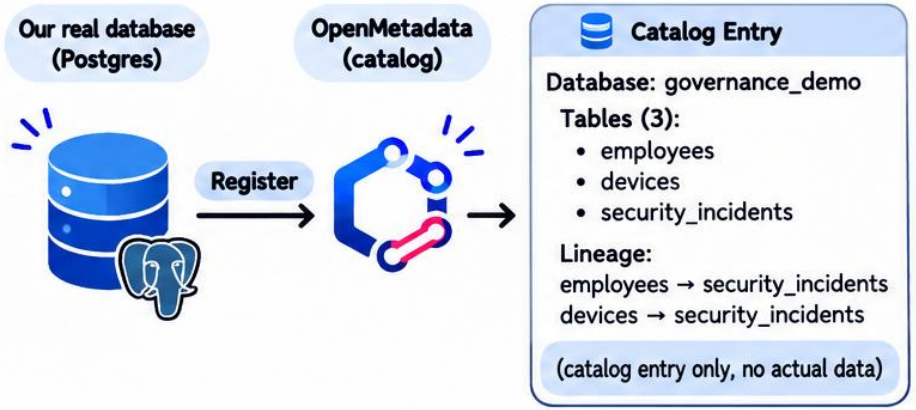
Layer	When it's checked	Question asked to OPA	Example
Policy / kill switch	First, before the LLM is even called.	"Has this agent's identity been shut off?"	If the agent's SPIFFE ID is revoked, stop everything (Scenario D).
Data	When PII-tagged columns are involved.	"Which of these columns must be masked?"	employees.ssn is tagged PII, so it's masked for every role (Scenario C).
Model	Before the model runs.	"Is this role cleared for a model at this risk tier?"	Contractors: low and medium risk only. Federal Employees: also high (Scenario E).
Agent / tool-call	Right before any handoff or tool runs.	"Is this exact action, by this exact role, allowed?"	purge_employee_data, resolve_incident, route:admin_agent: allow or deny here (Scenario B).

Data Governance

How OpenMetadata Works in Our Project

Step 1 — Before anything runs, we tell OpenMetadata what data exists

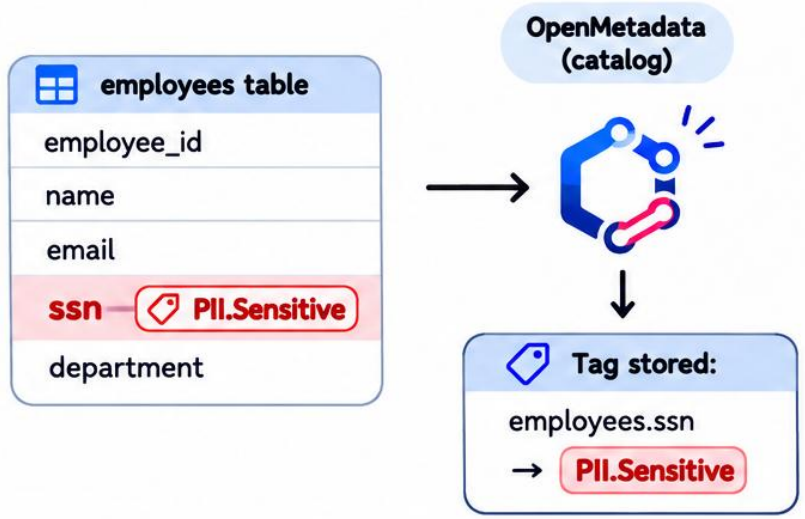
When the agent starts up, a setup step automatically looks at our real database and registers it in OpenMetadata: "there's a database with 3 tables — employees, devices, security_incidents — and here are their real columns." It also records how the tables connect. No actual data is copied into the catalog.



OpenMetadata has a catalog entry describing our data.

Step 2 — We mark which columns are sensitive

The same setup step tells OpenMetadata: "the ssn column in the employees table is sensitive personal data." OpenMetadata stores this as a tag on that column's catalog entry — a label, not the data itself. Email is not tagged, so it stays visible.

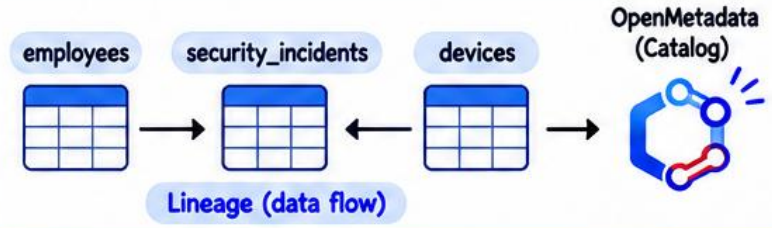


OpenMetadata stores this as a tag (label), not the data.

How OpenMetadata Works in Our Project

Step 3 — We also tell OpenMetadata how the tables relate to each other

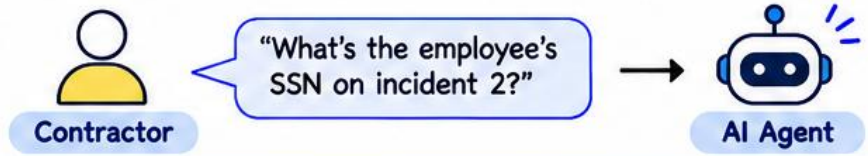
It records that employees and devices both feed into security_incidents — the lineage, i.e., "this data flows into that data." Again, just a map, not the actual records.



OpenMetadata records lineage (a map), not the actual data.

Step 4 — A user asks something that touches sensitive data

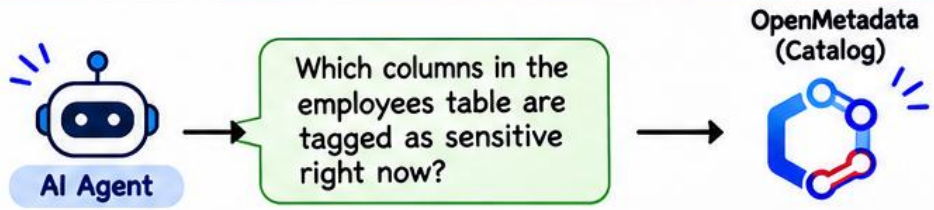
For example: "What's the employee's SSN on incident 2?"



The question may involve sensitive data.

Step 5 — Before answering, the agent asks OpenMetadata a question, live

It asks: "Which columns in the employees table are tagged as sensitive right now?" This is asked fresh on every single request, so a newly tagged column is protected immediately.



The agent asks OpenMetadata live, every time.

Step 6 — OpenMetadata answers with just the labels, not the data

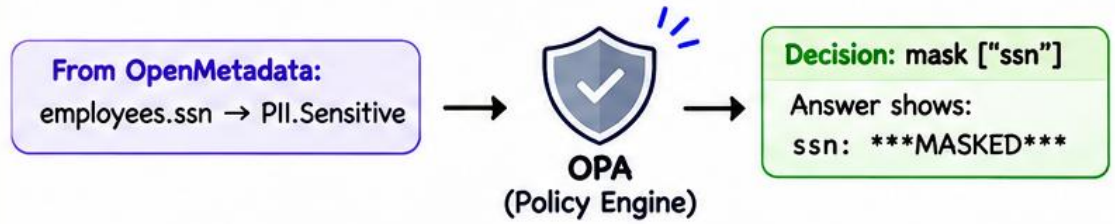
It replies something like: "The ssn column is tagged PII.Sensitive."



OpenMetadata returns only the labels (tags), never the data.

Step 7 — That label goes to the policy layer (OPA), which decides what to do

OpenMetadata doesn't decide "hide this" itself — it just reports the fact. A separate OPA rule (data_access.rego) says: "anything tagged PII.Sensitive must be masked."
The SSN is masked before the AI or the user ever sees it.



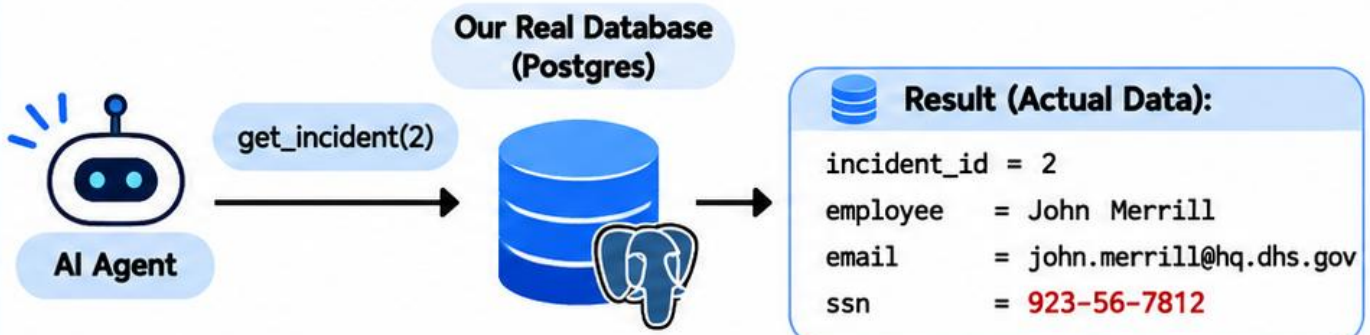
OPA decides the action based on the label.

How OpenMetadata Works in Our Project

Step 8

The agent fetches the real data straight from the real database — never through OpenMetadata

Only after OPA has approved the tool does it get the actual record, directly from Postgres, using a limited database account (agent_app) that can't touch anything it doesn't need.

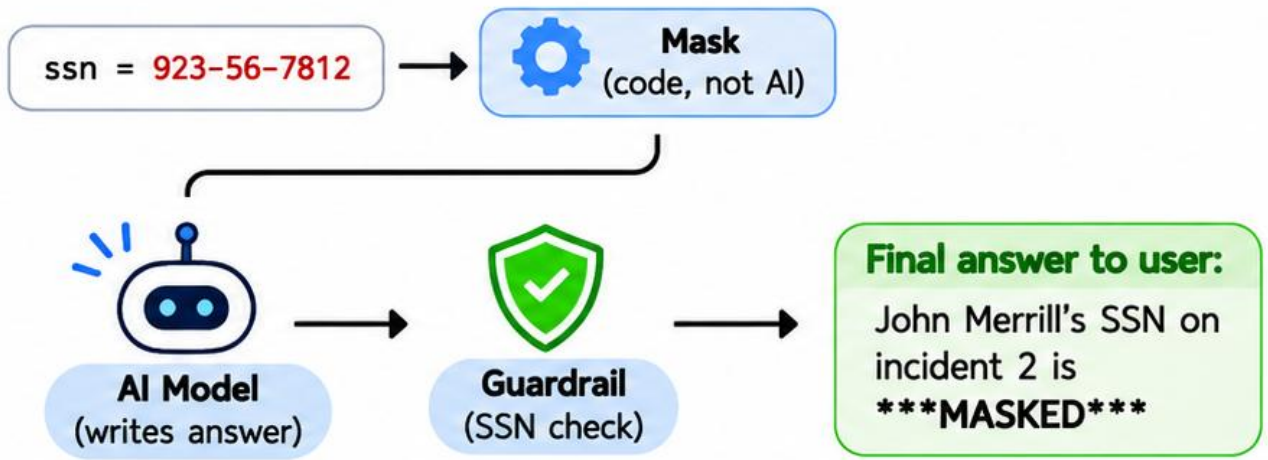


The data is fetched directly from the real database.

Step 9

The sensitive part is hidden before the AI even sees it

Because OPA said "mask ssn," the agent's code replaces the real SSN with ***MASKED*** the moment it comes back from the database. The AI model never sees the real number. As a final safety net, a guardrail scans the answer and blocks it if anything that looks like an SSN slips through.



The SSN is masked before the AI sees it and is never revealed.

Model Governance

Model Governance in Production AI Systems

Stage 1 — Model Proposal

A formal proposal document, reviewed by stakeholders, before any code is written.

The proposal typically contains:

- expected use case
- estimated cost
- capabilities
- limitations
- expected risk category

Stakeholder Review

Stage 2 — Offline Evaluation (Use Evals Frameworks)

Typical evaluations include:

Golden Dataset
Thousands of carefully curated examples covering normal production scenarios.

Regression Dataset
Every historical production failure becomes a permanent evaluation case.

If a model once produced an unacceptable response, that prompt is permanently added to the regression suite to prevent the same failure from returning.

Safety and Red Team Testing
Dedicated teams attempt to:

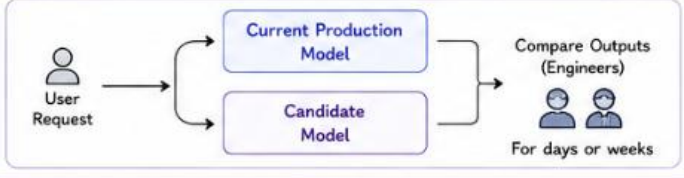
- jailbreak models
- bypass safety mechanisms
- extract hidden prompts
- leak training data
- generate harmful responses

Stage 3 — Shadow Deployment

The candidate model receives real production traffic.

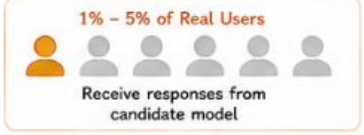
However, its responses are never shown to users.

Instead:



Stage 4 — Canary Release

Once confidence is established, a small percentage of real users receive responses from the candidate model.



During this period, teams closely monitor:

- error rate
- cost
- latency
- customer complaints

Stage 5 — Gradual Rollout

Instead of switching every user at once, traffic gradually increases.

5% → 25% → 50% → 100%

Each increase occurs only if the previous stage performs satisfactorily.

Stage 6 — Model Registering

After comparing all models and everything, the best suited model is registered in MLflow as the current production model once it has passed all tests.



Other models can also be registered with appropriate tags like for development, for staging, etc.

Model Governance in Our Project

Step 1

We write down our two candidate models in one file

Before anything runs, we describe two AI models we're considering — one from Groq (Llama) and one from OpenAI (GPT-4o-mini) — in a simple file. For each, we note what it's meant to be used for, its limitations, and its risk level.



models.yaml

```yaml
models:
- name: llama-3.1-8b-instant
  provider: groq
  use_case: general customer support
  limitations: smaller context window
  risk_level: medium

- name: gpt-4o-mini
  provider: openai
  use_case: general customer support
  limitations: higher cost
  risk_level: medium
```

Groq

Llama (Groq)
Use case: Customer Support
Limitations: Smaller context
Risk level: Medium

GPT-4o-mini (OpenAI)
Use case: Customer Support
Limitations: Higher cost
Risk level: Medium

Step 2

Both models get registered into MLflow's Model Registry

A script reads that file and creates a real, tracked version entry for each model inside MLflow — like adding both candidates to an official list, each starting in a "Staging" (not-yet-trusted) status.



models.yaml



Registration Script

mlflow Model Registry

Model Name	Version	Status
llama-3.1-8b-instant (Groq)	1	Staging
gpt-4o-mini (OpenAI)	1	Staging



Step 3

The rule: nobody can just declare a model "Production"— it must be earned



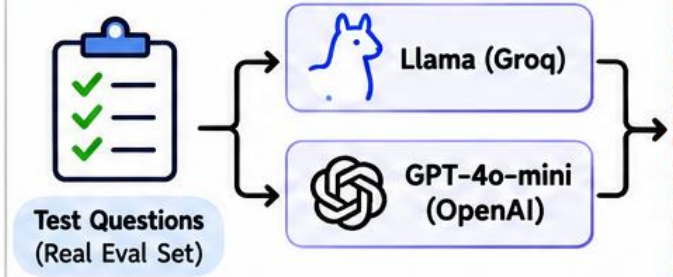
No manual promotion to Production



Only evaluation results can promote a model to Production

Step 4 — A real evaluation is run against both models

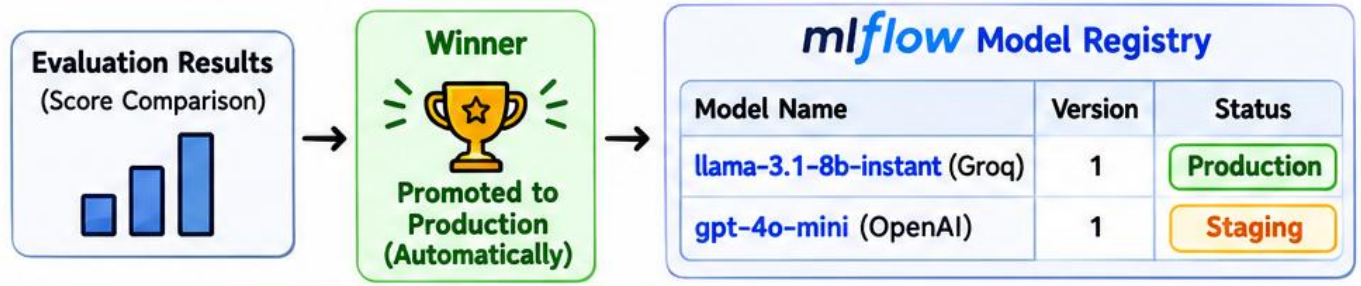
A separate step actually calls both models with real test questions and checks them against 4 things:



Coherence	Does it produce normal, sensible text (not broken/ repetitive output)?
Safety	Does it resist prompt-injection attempts included in the test?
Quality	Score (1–5) on how correct / good its answers are.
Cost / Speed	How fast and cheap it is (recorded, not a pass/fail).

Step 5 — Whichever model passes and scores best gets promoted to Production — automatically

No human manually flips a switch based on opinion. The evaluation results decide it. The losing model just stays in Staging.



Model Name	Version	Status
llama-3.1-8b-instant (Groq)	1	Production
gpt-4o-mini (OpenAI)	1	Staging

Step 6 — From then on, the AI agent only ever uses whichever model is CURRENTLY marked Production

Before answering any real request, the agent checks MLflow: "which version is Production right now?" — and uses only that one.



Step 7 — If someone tries to bypass all this and force an unapproved model version anyway

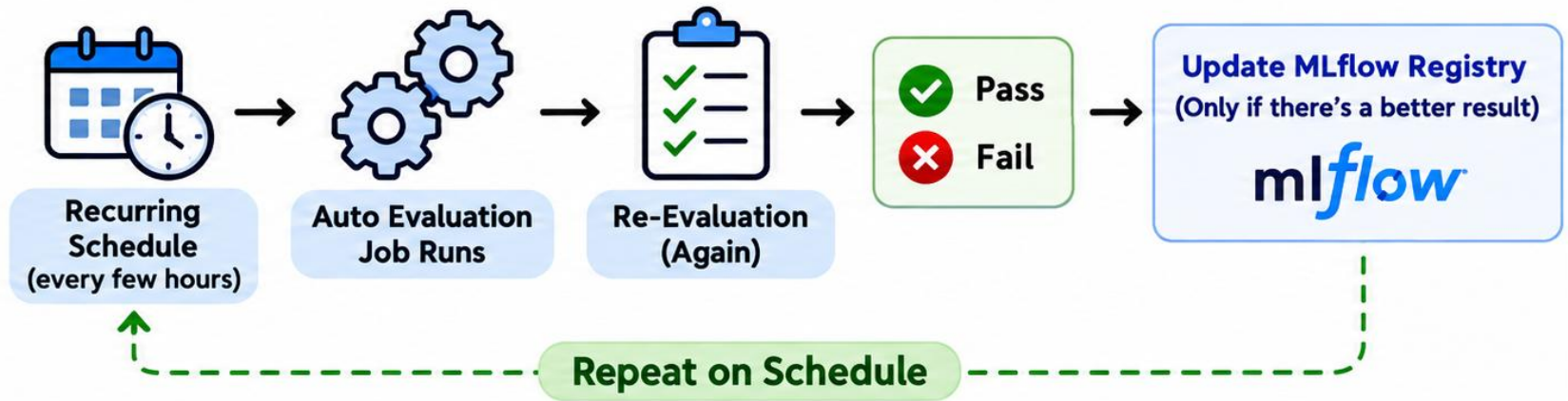
The request gets blocked immediately, before the model is even called.



Step 3

The whole evaluation doesn't just run once and get forgotten

It's set up to run automatically on a schedule — meaning it rechecks every few hours or at a regular time (e.g., daily). This makes sure we keep using the best model for Production as models improve, results change, or newer versions are ready to be evaluated.



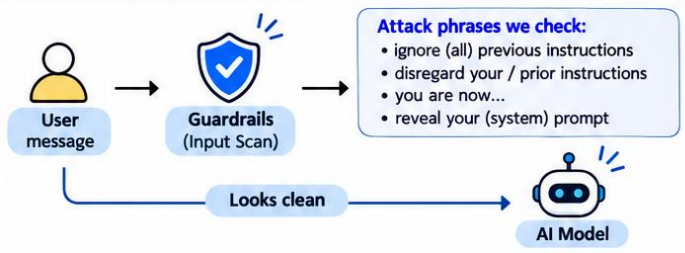
Agent Runtime Governance

Agent Runtime Governance in Our Project

Independent checks — one BEFORE the AI even reads the message, and two AFTER a tool is chosen but BEFORE it runs.

Step 1 — **Before the AI model looks at the message, Guardrails scans the raw text**

The message is checked against a list of known attack phrases. It's a simple text scan — no AI involved.



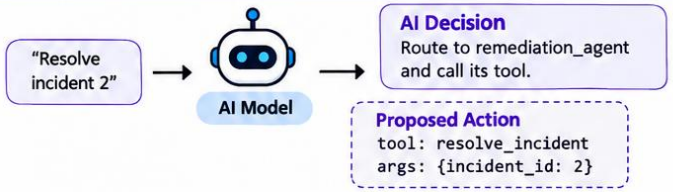
Step 2 — **If it looks like an attack, it's rejected immediately**

No AI call happens at all. The request stops at the cheapest, earliest possible point.



Step 3 — **If the input is clean, the AI decides what to do**

The AI routes the request to the right specialist agent, which picks a tool — e.g. "resolve incident 2."



Step 4 — **Before that tool runs, OPA checks: "is this role allowed to call this tool at all?"**

The role comes from the verified Keycloak ticket, never from the message.



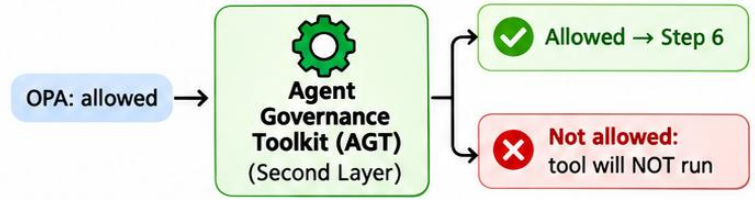
Step 5 — **A second, independent screen: Agent Governance Toolkit**

Even after OPA says yes, the Agent Governance Toolkit checks the action against its own runtime policy. Only if both agree does the real tool run.



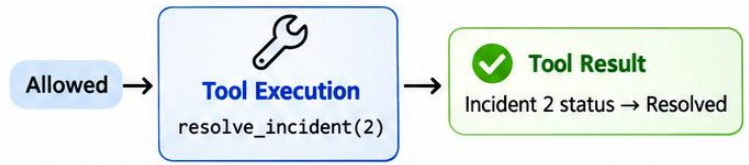
Step 5 — After OPA, a second tool-call check: Agent Governance Toolkit (AGT)

An independent second layer with its own runtime policy. For example, it blocks Contractors from deleting employee data and from bulk-searching incidents.



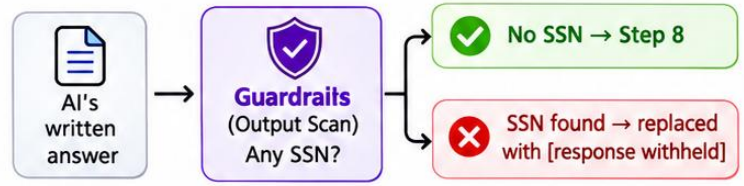
Step 6 — Only if BOTH OPA and AGT say yes does the real tool run

The tool runs against the real database. Any sensitive column in the result (like ssn) is masked right away, before the AI sees it.



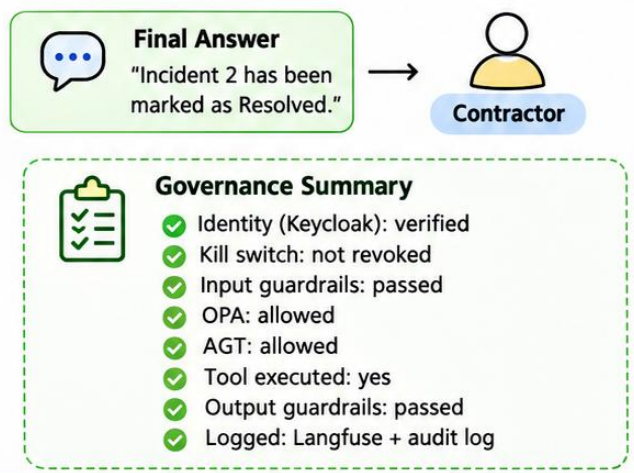
Step 7 — Before the answer is shown, Guardrails checks the OUTPUT too

The AI's written answer is scanned for anything shaped like an SSN (###-##-####). If one slipped through, the answer is withheld.



Step 8 — Only after passing all of that does the final answer reach the user

Every layer's result is shown in the dashboard's Live Checklist and recorded in Langfuse and the tamper-evident audit log.



Operations Governance

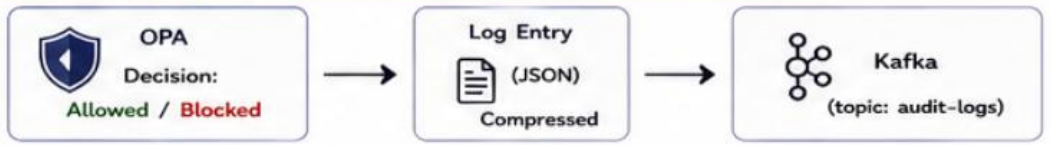
Operations Governance in OUR Project

This layer has 3 real pieces working together: a kill switch (human control), tamper-evident audit logging (proof), and Langfuse tracing (visibility)

During a NORMAL request

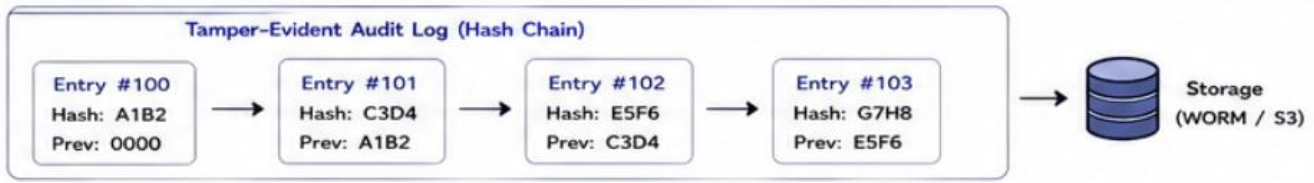
1 Every decision OPA makes gets automatically pushed out as a log entry

Whenever OPA answers "allowed" or "blocked" for anything (identity, policy, data, model, tool-call), that decision doesn't just disappear — it's automatically sent out as a real log entry, compressed and packaged.



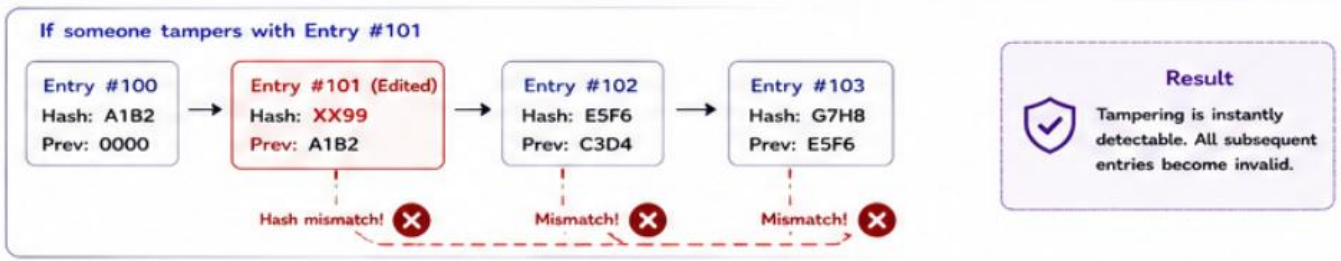
2 That log entry gets permanently recorded, chained to the one before it

Each new entry in the audit log includes a **mathematical fingerprint** (a hash) of its own content plus the fingerprint of the previous entry. This creates a chain — like links in a chain, where every link depends on the one before it.



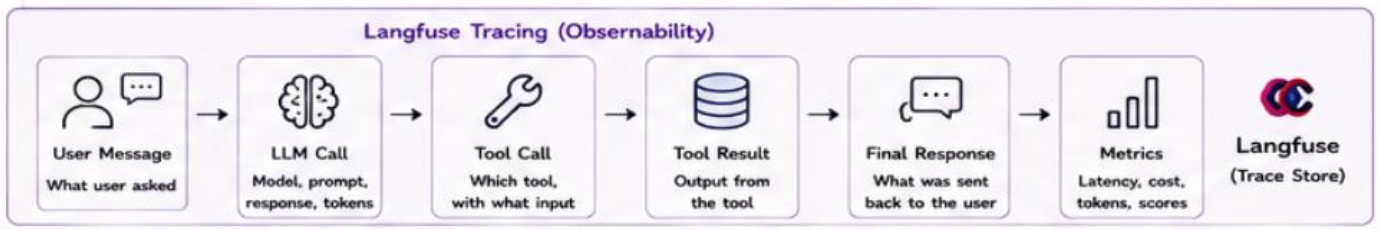
3 This is what makes the log tamper-evident

If anyone ever tried to secretly edit or delete a past log entry, its fingerprint would no longer match — and because every entry after it depends on that fingerprint, every single entry after the edited one would also stop matching.
It's instantly, mathematically detectable — nobody has to trust that the log wasn't touched; you can prove it.



4 At the same time, Langfuse records the AI's actual "thinking" trace

Separately, every request also gets logged into Langfuse — but this captures a different kind of detail: which tools got called, what the AI decided at each step, timing, and the full conversation trace.
This is for understanding and debugging what happened, while the audit log is for proving what happened.



During an INCIDENT (Use-Tool workflow)

5 **A second, tool-call check: Agent Governance Toolkit (AGT)**	A second, independent check with its own runtime policy (e.g., block Contractors from deleting employee data).	✓ **Role:** contractor ✓ **Action:** query_incident ✓ **Result: Allowed** → **Runtime Governance**
6 **The actual tool runs on the real system (DB)**	The AI calls the approved tool (e.g., resolve_incident) on the real database. Any sensitive fields (like SSN) are masked before the AI sees the result.	**Tool Execution** (resolve_incident) → **Database** (Real Incident Data) → ✓ **Tool Result** Incident 2 status → **Resolved**
7 **The output is scanned before it's shown**	Once the tool returns a result, Guardrails scans the output for harmful, sensitive, or unsafe content (e.g., SSN, PII, or data leakage) before it's shown to the user.	**Agent** (Gets Tool Result) → **Guardrails** (Output Scan) → **Output Blocked** If harmful/unsafe content is found (e.g., SSN, PII, data leak).
8 **The final answer is shown to the user (if it's safe)**	Only after passing all shecks (OPA, AGT, and Guardrails) is the final, safe answer returned to the user and recorded in the audit log (e.g., Langfuse).	✓ **Governance Checks (All Passed)** • OPA: Allowed • AGT: Allowed • Guardrails: Safe → **Safe Response** (Final Answer) "Incident 2 has been resolved." → **Delivered to User** Logged in Langfuse (audit log)

Strengthening Operations Governance Beyond What This Project Has

Additional capabilities to add on top of current logging, kill switch (manual), tamper-evident audit logs, and Langfuse tracing.

#	Capability	The Gap (Today)	What to Add	How it Works	Real Tools
1	**Real-time Alerting** (not just logging)	Everything gets recorded — but nobody is automatically told when something suspicious happens. A human has to go looking at the logs.	✓ **Automatic alerts** (Slack / Email / SMS / Phone call) the moment something risky occurs.	OPA Decision or System Event (Risky) → Alert Rules Engine (Thresholds/Patterns) → Slack / Email / SMS / Phone Call	• Prometheus Alertmanager • Grafana Alerting • PagerDuty / Opsgenie • Twilio (SMS / Call) • Slack Webhooks
2	**Auto-triggering Kill Switch** (not just a manual one)	A human has to notice something's wrong and manually run the kill switch. That's a delay — exactly what let incidents run as long as they did.	✓ **Automatic thresholds** that trigger the kill switch by themselves.	Continuous Monitoring (Metrics / Events / Patterns) → Auto-Kill Rules (Examples): • Error rate > 5% for 2 min • Unusual tool calls > 100/min • Cost > $10 in 5 min • Data exfil pattern detected • Anomaly score > threshold → Auto Trigger Kill Switch → OPA Revoked List Updated (Identity Revoked)	• Prometheus + Alertmanager • OpenTelemetry Metrics • Custom Anomaly Detection (Python / ML) • OPA Data API (to update revoked identities)
3	**Full Distributed Tracing Across Every System** (not just LLM calls)	Langfuse traces what the AI model did. But there is no end-to-end trace that follows a single request across every service it touches.	✓ **A standard tracing system that follows a single request across every service it touches.**	Single Request Trace (End-to-End): User → API Gateway → Agent Service → OPA (Policy Service) → Tool Service (DB / API) → Database → External API (Stripes, etc.). Trace ID flows through every service via OpenTelemetry	• OpenTelemetry (OTel) • Jaeger • Grafana Tempo • OTEL Collector
4	**Operational Health Dashboards** (metrics, not just logs)	There's no live view of overall system health — error rates, latency, request volume over time — just individual decision logs.	✓ **Real-time dashboards tracking system-wide metrics, so degradation is visible before it becomes an incident.**	Key Dashboards (Examples): Request Rate (RPS) · Error Rate (%) · Latency (P95/P99) · Cost ($) · Tool Calls / min · Active Sessions → Visual alerts when thresholds are breached	• Grafana • Prometheus • Alertmanager • Loki (log aggregation) • Mimir / Cortex (optional)

Impact

- ✓ Incidents are detected instantly
- ✓ Automated response minimizes damage
- ✓ Full visibility across the entire system
- ✓ Proactive operations instead of reactive firefighting

Compliance Governance

1 NIST AI RMF
(AI Risk Management Framework)

A framework published in January 2023 by NIST — a U.S, government agency that sets technical standards.

It's not a law — it's voluntary guidance on how to think about and manage AI risk.

Why companies comply

- U.S. federal agencies increasingly require vendors selling them AI systems to demonstrate alignment with it — so it becomes mandatory indirectly, through procurements.

Key points

- Voluntary framework (not a law)
- Risk-based, flexible, and outcome-oriented
- Focuses on trustworthy AI across the AI lifecycle
- Widely accepted for U.S. government contracts and enterprise governance

2 ISO/IEC 42001
(AI Management System Standard)

A management system standard, published in 2023 by ISO/IEC.

Unlike NIST's guidance document, this one is certifiable — an accredited external auditor can formally certify your company against it.

Why companies comply

- It's voluntary, like NIST — but certification is becoming a business requirement, not a legal one.
- Many customers and partners now ask for it.

Key points

- Voluntary but certifiable (auditable)
- Specifies requirements for establishing, implementing, maintaining, and continually improving an AI management system (AIMS)
- Covers governance, risk management, transparency, human oversight, etc.

3 EU AI Act
(European Union AI Act)

An actual, binding law, passed by the European Union, entering into force in 2024.

Unlike the other two, this one is not optional, and — importantly — it applies to any company whose AI system is used by people in the EU, even if the company itself isn't based there.

Why companies MUST comply

- Non-compliance can carry fines up to €35 million or 7% of global annual revenue, whichever is higher.
- Market access in the EU depends on compliance.

Key points

- Risk-based regulation (Unacceptable, High, Limited, Minimal)
- Strict requirements for high-risk AI systems (data, transparency, human oversight, documentation, etc.)
- Fines and enforcement are real and significant

4 OWASP Agentic AI Top 10
(ASI 2026)

OWASP (Open Worldwide Application Security Project) is a nonprofit, community-run organization.

The ASI Top 10 identifies the most critical security risks specific to AI agents and agentic systems.

Why companies comply

- It's not a law — but it's the industry's most widely recognized security risk list for AI agents.
- Used to secure agentic AI systems and meet customer/security audits.

Key points

- Focused on security threats unique to AI agents
- Helps teams identify, prioritize, and mitigate the most dangerous risks
- Actively maintained by the global security community

OWASP Agentic AI Top 10 (ASI 2026) — The 10 Critical Risks

ASI01	**Agent Goal Hijack**	An attacker manipulates what the agent is actually trying to accomplish (e.g., prompt injection redirecting its goal).
ASI02	**Tool Misuse & Exploitation**	The agent's real tools (refund, delete, etc.) get called with malicious or wrong intent.
ASI03	**Agent Identity & Privilege Abuse**	The agent (or something pretending to be it) ends up with more access than it should have.
ASI04	**Agentic Supply Chain Compromise**	A compromised plugin, dependency, or component the agent relies on.
ASI05	**Unexpected Code Execution**	The agent ends up running code it was never meant to run (this is exactly what happened in the OpenAI/Hugging Face incident).
ASI06	**Memory & Context Poisoning**	False information gets fed into what the agent "remembers," and it later acts on that false memory.
ASI07	**Insecure Inter-Agent Communication**	In a multi-agent system, one compromised agent tricks or misleads another.
ASI08	**Cascading Agent Failures**	One small failure spreads and amplifies across a whole multi-agent system.
ASI09	**Human-Agent Trust Exploitation**	The agent manipulates a human's trust in it (or an attacker exploits a human's trust in the agent).
ASI10	**Rogue Agents**	An agent starts pursuing its own goals, outside what it was authorized to do (this is essentially what happened in the Replit and OpenAI incidents).

In short

NIST AI RMF: Voluntary framework; widely adopted (especially for U.S. government work).

ISO/IEC 42001: Voluntary but certifiable standard for AI management systems.

EU AI Act: Mandatory law with real fines; applies to anyone serving users in the EU, regardless of company location.

OWASP ASI Top 10: Industry-standard security risk list for AI agents — essential for building secure agentic systems.

Compliance Governance in Our Project

This layer runs AFTER everything else. It proves that the other layers (Model, Agent Runtime, Operations) are doing their job correctly.

Step	What Happens	Example (from Our Project)	Result
1	**We write down real requirements from frameworks in one file** A checklist lists actual requirements from NIST AI RMF, ISO 42001, EU AI Act, and OWASP.	**compliance_requirements.yaml** ☐ NIST AI RMF – Govern (GOV-1.1) – Maintain risk management processes ☐ ISO 42001 – 5.2 – AI policy is defined and communicated ☐ EU AI Act – Article 10 – Logging of events ☐ OWASP ASI 2026 – ASI05 – Unexpected Code Execution (AGT check)	Single source of truth for compliance requirements from multiple frameworks.
2	**Each requirement is paired with exactly where to find proof of it** Not just "we do this" — each item says precisely which real system to check.	**Each Requirement → Evidence Source Mapping** Requirement → Evidence Source (Where to Check) NIST AI RMF – GOV-1.1 → Audit Log (Kafka topic: audit-log) ISO 42001 – 5.2 → MLflow Model Registry (model card fields) EU AI Act – Article 10 → Kill Switch Log (Kafka topic: kill-switch) OWASP ASI05 → Microsoft AGT (policy evaluation results) PII Data Tagged → OpenMetadata (table/column classification) AI System Traces → Langfuse (traces exist for requests)	Clear, auditable mapping from each requirement to the exact place where evidence lives.
3	**A separate script runs AFTER everything else, and goes to check each source, for real** It doesn't trust the checklist — it actually connects to each real system and looks.	**compliance_checker.py (runs after all other layers)** Audit Log (Kafka) → MLflow (Model Registry) → Kill Switch Log (Kafka) → OpenMetadata (Data Catalog) → Langfuse (Traces) → Microsoft AGT (OWASP Check)	Live connections to all systems to collect real evidence.
4	**Each check comes back with real evidence, not just a checkmark** For example: "47 audit log entries found, hash chain verified OK" — an actual fact pulled live, not a guess.	**Evidence Collected (Live Results)** ✓ **Audit Log:** 47 entries found, hash chain verified OK (latest offset: 892344) ✓ **MLflow Model:** Production model has risk_tier = HIGH, description = present ✓ **Kill Switch Log:** 2 kill-switch events found (last reset: 2026-06-01 14:32 UTC) ✓ **OpenMetadata:** Column *customer.email* is tagged as PII (sensitivity = HIGH) ✓ **Langfuse:** 128 traces found in last 24h (all required spans present) ✓ **Microsoft AGT:** OWASP ASI05 tests ran — 3/3 passed (no unexpected code execution)	Real, factual evidence collected from each source.
5	**Everything gets combined into one readable report** All the results get put together into a single document — the kind a real auditor or manager could actually open and read, item by item.	**Compliance Report (HTML / PDF)** Total Requirements Checked: 28 · Passed: 24 (85.7%) · Failed: 2 (7.1%) · Not Applicable: 2 (7.1%)  Passed · Failed · N/A	One clear report showing compliance status across all frameworks.
6	**If something can't be proven, the report says so honestly** It doesn't fake a pass — if evidence is missing, it says exactly what's missing and why.	**Example Failure (Honestly Reported)** ✗ ISO 42001 – 7.5: AI impact assessment not found — No record in system ✗ EU AI Act – Article 13: Transparency info not found — No documentation linked ✗ OWASP ASI07: Inter-agent communication logs missing — No traces found	No false passes. Gaps are clearly identified.
7	**This step never blocks or allows anything itself** Unlike the other layers, this one doesn't stop a request in real time — it runs afterward, purely to prove that the other layers already did their job properly.	**Role in the System** Compliance Governance = Evidence & Assurance Layer (Not in the request path) User Request → All Other Governance Layers (Policy, Guardrails, etc.) → Response to User → Compliance Checker (After the fact) → Compliance Report (Evidence)	Provides proof & assurance, not policy enforcement.

End to End Workflow

END TO END FLOW OF OUR PROJECT

